

# Chat with Data

## Data-Egress & Third-Party Access Audit

RAM Pilot edition (repo: chat\_with\_data\_ram\_Pilot)

### **VERDICT: PASS**

After two hardening fixes applied during the audit, the application contacts exactly one external service: the company-internal LiteLLM gateway. No user data can reach any other cloud service or third party.

AI Solution Architect: Thiago Alvares

Prepared July 2026 · Document Version 1.0

## Contents

# 1. Executive Summary

This audit examined every network path in the Chat with Data RAM Pilot application to answer one question: can data loaded into the app ever reach a cloud service or third party other than the provided LiteLLM API? The answer, after remediation, is no.

- The Python service contains exactly one network call site (`services/llm.py`), and it targets only the configured LiteLLM gateway URL. No other module in the service imports any network-capable library.
- The .NET front end makes outbound calls only to the Python service on localhost.
- Application data lives exclusively in server RAM. Nothing is written to a database or cloud storage; the only local writes are application log files. Exports (Word, Excel, PNG) stream directly to the requesting user's browser.
- Two third-party connections were found in the browser front end — Google Fonts and the Chart.js CDN. Neither carried application data, but both were removed during the audit by self-hosting the assets. The application now loads with zero third-party requests.
- A live browser network capture during a full feature exercise confirmed: every request goes to the application host only.

Defense-in-depth recommendations for the production server (a firewall egress allowlist and TLS certificate verification at the gateway) are listed in Section 8. None are blockers.

## 2. Scope and Method

### 2.1 What was audited

Repository `chat_with_data_ram_Pilot` (the version intended for deployment): the Python micro-service (all modules), the .NET front end (all code and configuration), the browser UI (`index.html` and every asset it loads), Docker/compose files, environment templates, and dependency manifests.

### 2.2 Method

1. Static review — every source file was searched for network-capable imports (`requests`, `urllib`, `socket`, `http clients`, `WebClient/HttpClient`) and for any `http/https` URL. Each hit was traced to its destination and purpose.
2. Data-at-rest review — where uploaded data lives, what is written to disk, what the logs contain, and where exports go.
3. Dynamic verification — the application was run end-to-end (upload, question, refine, all four export types, health checks) with the browser's full network log captured. Every request was inspected for its destination host.
4. Dependency review — Python packages and NuGet packages were checked for known runtime telemetry (none present).

### 3. Data-Flow Map

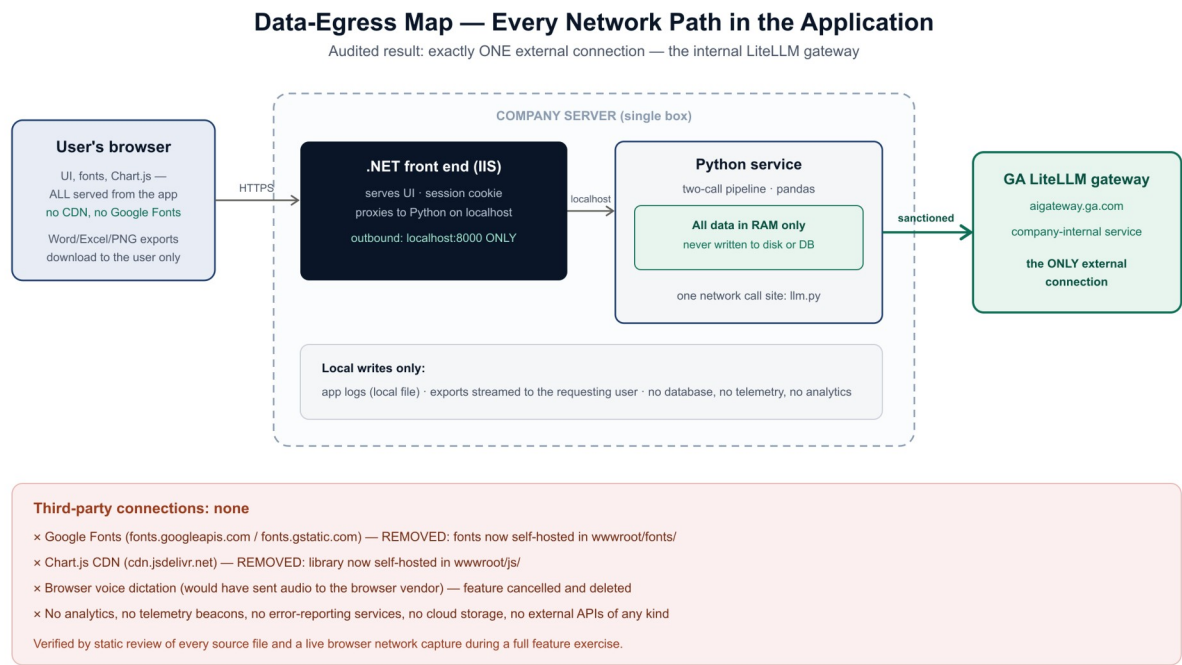


Figure 1 — Every network path in the application. The single sanctioned egress is the internal LiteLLM gateway.

Uploaded files travel: browser → .NET (localhost proxy) → Python, where they are parsed by pandas and held in the Python process memory, keyed by session. Questions run the two-call pipeline; only the pipeline's model calls leave the server, and only toward the gateway.

### 4. Complete Inventory of Network Touchpoints

Every place in the codebase that can create a network connection, with its destination and audit verdict:

| Component                                      | Destination                                   | Data carried                                                     | Verdict                       |
|------------------------------------------------|-----------------------------------------------|------------------------------------------------------------------|-------------------------------|
| services/llm.py (the ONLY Python network call) | LITELLM_API_BASE (GA gateway)                 | Prompts: schema, question, history, computed results (Section 5) | <b>SANCTIONED</b>             |
| .NET HttpClient (Program.cs)                   | PythonService.BaseUrl = http://localhost:8000 | Proxied requests, same box                                       | <b>INTERNAL ONLY</b>          |
| Browser → .NET                                 | The application host (IIS)                    | UI, uploads, answers, exports                                    | <b>INTERNAL ONLY</b>          |
| Browser assets (fonts, Chart.js, logo)         | Self-hosted under wwwroot/                    | None — static files                                              | <b>LOCAL (fixed in audit)</b> |
| mock_llm.py (test tool only)                   | localhost — never deployed as part of the app | Test traffic                                                     | <b>LOCAL / TEST ONLY</b>      |
| Logging (logger.py)                            | Local log file on the server                  | Timestamps, question                                             | <b>LOCAL WRITE</b>            |

| Component                   | Destination                                  | Data carried           | Verdict              |
|-----------------------------|----------------------------------------------|------------------------|----------------------|
|                             |                                              | snippets               |                      |
| Exports<br>(Word/Excel/PNG) | Streamed to the<br>requesting user's browser | The user's own results | <b>USER DOWNLOAD</b> |

No analytics, telemetry beacons, error-reporting services (e.g. Sentry/App Insights), cloud storage SDKs, or third-party APIs exist anywhere in the codebase.

## 5. What Is Sent to the LiteLLM Gateway (the Sanctioned Path)

For transparency, this is precisely what the two model calls transmit to the gateway — and it does include excerpts of the loaded data, which is inherent to how the application works:

| Sent on                             | Content                                                                                                                                                                                                                   |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Call 1 (code generation)            | The data schema: column names, types, unique counts, up to 3 sample values per column, and the first 3 rows; the user's question; recent conversation history; a preview (up to 1,000 characters) of the previous result. |
| Call 2 (answer writing)             | The computed result of the executed pandas code (as text) plus result metadata (row/column counts).                                                                                                                       |
| Refine (one sentence / more detail) | The same stored computed result — re-sent to the gateway with a different phrasing instruction. No new data.                                                                                                              |

The full raw file is never transmitted — only the schema/sample excerpt and computed results. All of it goes exclusively to the gateway configured in LITELLM\_API\_BASE, authenticated with the gateway API key.

## 6. Findings and Remediations

### 6.1 F1 — Google Fonts (fixed)

The UI loaded the Inter and IBM Plex Mono fonts from fonts.googleapis.com / fonts.gstatic.com. No application data was sent, but each user's browser contacted Google (transmitting IP address and browser metadata), and the app would degrade on an isolated network.

Remediation: the font files (latin subsets, 7 files) are now self-hosted under wwwroot/fonts/ with a local stylesheet. Appearance is unchanged.

### 6.2 F2 — Chart.js CDN (fixed)

The charting library was loaded from cdn.jsdelivr.net. Same class of issue as F1 — no application data, but a third-party contact and an external dependency. Remediation: Chart.js 4.4.0 is now self-hosted at wwwroot/js/chart.umd.min.js.

### 6.3 F3 — Voice dictation (removed before the audit)

A voice-input feature built during development would have used the browser's speech service (audio sent to the browser vendor's cloud). It was cancelled on request and fully deleted; the audit confirms no trace remains.

## 6.4 Result after remediation

The application makes zero third-party requests. This was verified live: with the app fully exercised (upload, question, refine, Excel export, raw-data export, Word report export, health checks), the browser's network log showed every request going to the application host only (Section 9).

## 7. Data at Rest, Logs, and Exports

- In-RAM by design: uploaded DataFrames, conversation history, and results live only in the Python process memory and vanish on restart. The RAM Pilot never writes user data to disk or to any database.
- Logs: the service writes a local rotating log file. Entries include timestamps, session ids, and the first ~80 characters of each question — treat log files with normal server-access controls.
- Exports: Word/Excel/PNG files are generated on the server in memory and streamed to the requesting user's browser download. The server keeps no copy.
- The storage module also contains optional disk/SQL Server backends (unused in this edition; STORAGE\_BACKEND=memory). If ever enabled, they write only to the server's own disk or the company SQL Server — still no third party.

## 8. Residual Considerations and Recommendations

Nothing below is a third-party data path today. These are defense-in-depth measures for the production server:

### 8.1 Add a firewall egress allowlist (recommended)

The pipeline executes model-generated pandas code on the server. That code comes only from the trusted internal gateway and the prompts constrain it to pandas, but executed Python could in principle import a network library. A server firewall rule allowing outbound traffic ONLY to the LiteLLM gateway host makes 'nothing else can ever be contacted' a property of the infrastructure, not just the code. This is the single most valuable hardening step.

### 8.2 TLS certificate verification on gateway calls (observation)

llm.py calls the gateway with certificate verification disabled (verify=False), carried over verbatim from the validated prototype (likely due to an internal CA). This does not create a third-party data path, but enabling verification against the company CA bundle at deployment would prevent a hypothetical man-in-the-middle on the internal network. Because this file is part of the frozen analytical core, the recommended change is a deployment decision, not applied in this audit.

### 8.3 Build-time (not runtime) external contacts

Installing dependencies (pip/NuGet) and building Docker images contacts public package registries from the build machine — standard software supply chain, no application data involved. Optionally set DOTNET\_CLI\_TELEMETRY\_OPTOUT=1 on build machines to silence the .NET SDK's build-time telemetry (the deployed app itself sends none).

## 8.4 Keep assets vendored

Future UI work should keep fonts/libraries self-hosted — no new CDN links or external scripts. The repository guidance file (CLAUDE.md) records this rule.

# 9. Verification Evidence

## 9.1 Static sweep

Search of every .py/.cs/.html/.json/.yaml/.sh/.ps1 file for network imports and URLs found: one requests import (llm.py), one call site (requests.post to the configured gateway), and — before remediation — the two front-end CDN references, since removed. No socket, urllib, httpx, aiohttp, smtplib, or WebRequest usage anywhere.

## 9.2 Live network capture (after remediation)

Full feature exercise with the browser network log recording. Every request and its destination:

| Request (in order)                                   | Destination      |
|------------------------------------------------------|------------------|
| GET / (the app page)                                 | application host |
| GET /fonts/fonts.css + 6 x .woff2 font files         | application host |
| GET /js/chart.umd.min.js                             | application host |
| GET /images/ga_logo.png                              | application host |
| POST /upload/standard · POST /ask · POST /ask/refine | application host |
| GET /export/last_result · GET /export/debug_result   | application host |
| POST /export/conversation · GET /healthz             | application host |

**External hosts contacted: none. Server-side, the Python service's only outbound call went to the configured gateway address.**

# 10. Conclusion

The architecture is intrinsically well-suited to the requirement: one deliberate egress point, everything else local. The two CDN conveniences inherited from the prototype were the only third-party contacts, they carried no application data, and they have been eliminated by self-hosting. As deployed from this repository, data loaded into Chat with Data can reach exactly one external service — the company's LiteLLM gateway — and nothing else. Applying the Section 8.1 firewall allowlist at go-live will enforce that guarantee at the network level as well.